ВІННИЦЬКИЙ НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ

ІНСТИТУТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ ТА КОМП'ЮТЕРНОЇ ІНЖЕНЕРІЇ

КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Дослідження алгоритмів сортування

Звіт про виконання лабораторної роботи № 4

Виконав (ла): ст. 2 курсу, групи 2ПІ-25Б

Коваль Олександр

Перевірив: викладач Карась О.В.

Вінниця 2026

Тема: Дослідження алгоритмів сортування. Сортування вставкою та сортування злиттям.

Мета: набути навичок реалізації та аналізу алгоритмів сортування мовою C++; експериментально порівняти час виконання двох алгоритмів на однакових вхідних масивах та перевірити відповідність отриманих результатів теоретичним оцінкам.

ЗАВДАННЯ

Програмно реалізувати два алгоритми сортування: № 3 — сортування вставкою (Insertion sort) та № 8 — сортування злиттям (Merge sort).

Порівняти між собою час їх виконання на однакових випадкових масивах різної розмірності.

Зробити висновок щодо відповідності отриманих результатів заявленим теоретичним оцінкам часу виконання.

КОД ПРОГРАМИ

```
#include <iostream>

#include <vector>

#include <chrono>

#include <random>

#include <algorithm>



using namespace std;

using namespace chrono;









void insertionSort(vector<int>& a) {

    for (int i = 1; i < (int)a.size(); i++) {

        int key = a[i];

        int j = i;
```

```
        while (j > 0 && a[j - 1] > key) {
            a[j] = a[j - 1];
            j--;
        }
        a[j] = key;
    }
}
```

```
void mergeSort(vector<int>& a, int left, int right) {
    if (right - left <= 1) return;
    int mid = left + (right - left) / 2;
    mergeSort(a, left, mid);
    mergeSort(a, mid, right);

    vector<int> tmp(a.begin() + left, a.begin() + right);
    int n = right - left;
    int m = mid - left;
    int i = 0;
    int j = m;
    for (int k = 0; k < n; k++) {
        if (i < m && (j >= n || tmp[i] <= tmp[j]))
            a[left + k] = tmp[i++];
        else
            a[left + k] = tmp[j++];
    }
}
```

```
vector<int> makeArray(int n) {
    mt19937 gen(12345);
    uniform_int_distribution<int> dist(0, 1000000000);
    vector<int> a(n);
    for (int i = 0; i < n; i++) a[i] = dist(gen);
    return a;
}

int main() {
    cout << "n\tInsertion (ms)\tMerge (ms)\n";

    vector<int> a1;
    vector<int> a2;

    for (int n : {10000, 100000}) {
        a1 = makeArray(n);
        a2 = a1;

        auto t1 = high_resolution_clock::now();
        insertionSort(a1);
        auto t2 = high_resolution_clock::now();
        mergeSort(a2, 0, n);
        auto t3 = high_resolution_clock::now();

        double ms1 = duration<double, milli>(t2 - t1).count();
        double ms2 = duration<double, milli>(t3 - t2).count();
```

```
        if (!is_sorted(a1.begin(), a1.end()) || !is_sorted(a2.begin(), a2.end())) {
            cout << "ПОМИЛКА: масив не відсортовано!" << endl;
            return 1;
        }

        cout << n << "\t" << ms1 << "\t\t" << ms2 << "\n";
    }

    cout << "\nПісля сортування вставкою (перші 20): ";
    for (int i = 0; i < 20 && i < (int)a1.size(); i++)
        cout << a1[i] << " ";
    cout << endl;

    cout << "\nПісля сортування злиттям (перші 20): ";
    for (int i = 0; i < 20 && i < (int)a2.size(); i++)
        cout << a2[i] << " ";
    cout << endl;

    cout << "\nНатисніть Enter..." << flush;
    cin.get();

    return 0;
}
```

РЕЗУЛЬТАТИ ТЕСТУВАННЯ ТА РОБОТИ ПРОГРАМИ

Вимірювання проведено на одному комп'ютері, компілятор g++ 14.2, прапорець -O2. Для обох алгоритмів використано однакові випадкові масиви з однаковим зерном генератора, щоб забезпечити коректність порівняння. Час вимірюється функцією high_resolution_clock.

Отримані результати:

n = 10000 — сортування вставкою: близько 22 мс, сортування злиттям: близько 2,3 мс;

n = 100000 — сортування вставкою: близько 2400 мс, сортування злиттям: близько 29 мс.

Вивід програми:

```
n      Insertion (ms)      Merge (ms)
10000  23.1128             2.7654
100000 2274.33             29.258

Після сортування вставкою (перші 20): 858 30925 44200 46131 46983 55667 63470 69971
93415 107251 117074 123641 126720 150300 158922 172740 174954 179152 225068 233684

Після сортування злиттям (перші 20): 858 30925 44200 46131 46983 55667 63470 69971
93415 107251 117074 123641 126720 150300 158922 172740 174954 179152 225068 233684

Натисніть Enter...
```

БЛОК-СХЕМА АЛГОРИТМУ СОРТУВАННЯ ВСТАВКОЮ (ГОСТ)

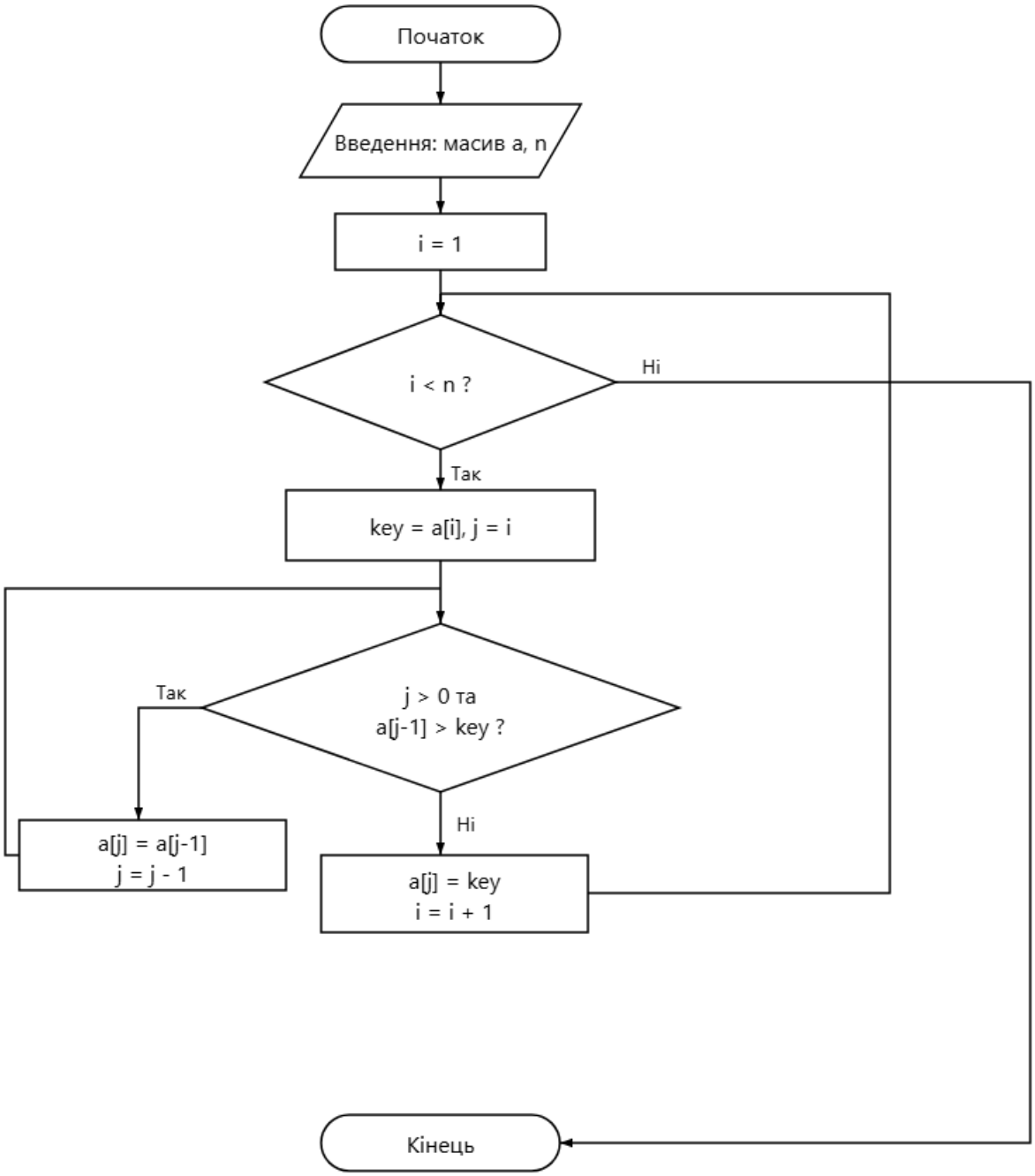


Рисунок 1 — Сортування вставкою

Вінниця 2026

БЛОК-СХЕМА АЛГОРИТМУ СОРТУВАННЯ ЗЛИТТЯМ (ГОСТ)

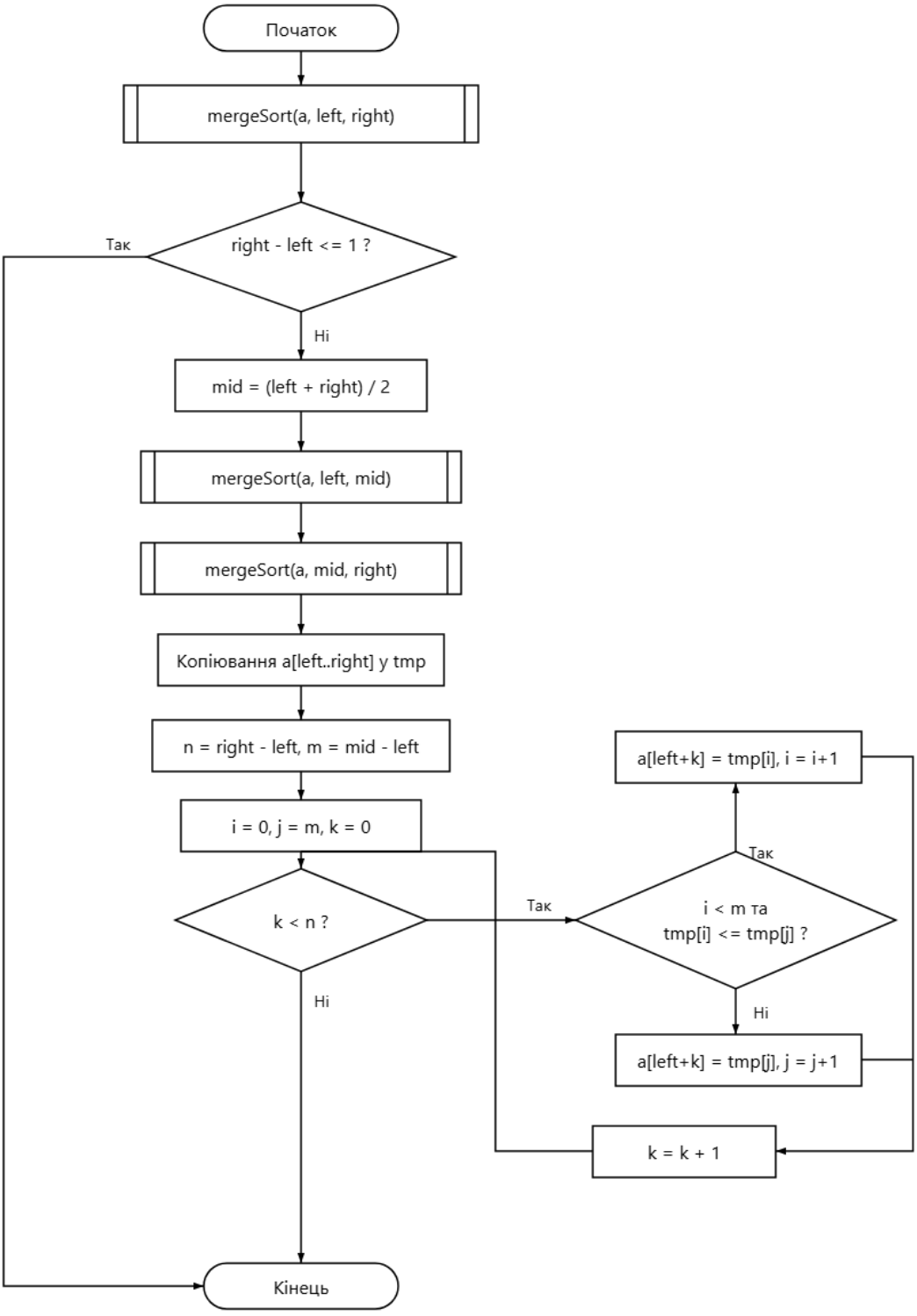


Рисунок 2 — Сортування злиттям

Вінниця 2026

Висновок: У результаті виконання лабораторної роботи реалізовано два алгоритми сортування: сортування вставкою та сортування злиттям. Програма вимірює час виконання кожного алгоритму на однакових випадкових масивах та перевіряє коректність результату функцією is_sorted. Отримані результати відповідають заявленим теоретичним оцінкам: при збільшенні n у 10 разів час сортування вставкою зріс приблизно у 100 разів, що підтверджує оцінку $O(n^2)$, а час сортування злиттям зріс лише приблизно у 12 разів, що відповідає оцінці $O(n \log n)$. На $n = 100000$ сортування злиттям виявилося приблизно у 85 разів швидшим за сортування вставкою. Реалізація використовує лише стандартні бібліотеки iostream, vector, chrono, random та algorithm, прості функції та цикли; окремо перевірено збіжність результатів двох алгоритмів на однакових даних.

Вінниця 2026